Επεξεργασία Δεδομένων Ροής με Kafka, Spark και MongoDB

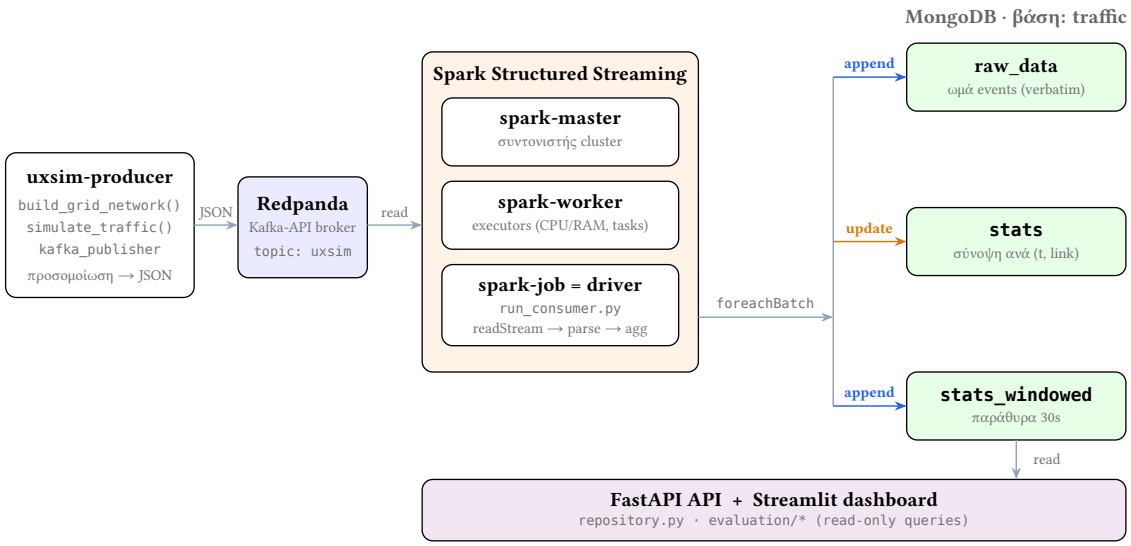
Project Εαρινού Εξαμήνου 2026 | Συστήματα Διαχείρισης Μεγάλων Δεδομένων
Τμήμα Μηχανικών Η/Υ και Πληροφορικής, Πανεπιστήμιο Πατρών
Βασίλειος Μπίτζας | Μάιος 2026
MachineGuid: `dde89f75-f5fd-4256-b226-cab61dbc0161`

1. Εισαγωγή

Η εργασία υλοποιεί ένα ολοκληρωμένο pipeline ανάλυσης ροής δεδομένων κυκλοφορίας. Δεδομένα οχημάτων που παράγονται από τον προσομοιωτή UXSIM αποστέλλονται σε έναν Kafka-συμβατό broker (Redpanda), καταναλώνονται σε πραγματικό χρόνο από Spark Structured Streaming και αποθηκεύονται τόσο σε ακατέργαστη όσο και σε συγκεντρωτική μορφή σε MongoDB. Όλη η στοίβα τρέχει τοπικά μέσω Docker Compose με μία εντολή (`docker compose up -d`) για τα 6 βασικά services, ή με ενεργοποίηση όλων των profiles ώστε να σηκωθούν επιπλέον το REST API και το dashboard:
`docker compose --profile api --profile dashboard up -d`

Η παρούσα αναφορά καλύπτει τα τρία ζητούμενα ερωτήματα της εκφώνησης (παραγωγή προς Redpanda, επεξεργασία με Spark, αποθήκευση σε MongoDB) και επιπλέον τρία bonus που υλοποιήθηκαν: (i) windowed aggregations 30 δευτερολέπτων με min/max ταχύτητα ανά link, (ii) REST API με FastAPI πάνω από τη MongoDB και (iii) Streamlit dashboard με ζωντανό χάρτη δικτύου σε Plotly.

2. Αρχιτεκτονική Συστήματος



Σχήμα 1: Ροή δεδομένων από τον προσομοιωτή ως την παρουσίαση. Όλα τα κουτιά είναι ανεξάρτητα containers στο ίδιο bridge network (`dsnet`).

Η στοίβα αποτελείται από έξι default containers και δύο προαιρετικά πίσω από Docker Compose profiles (Πίνακας 1). Όλες οι υπηρεσίες επικοινωνούν μέσω του εσωτερικού bridge δικτύου `dsnet` και αναζητούνται με DNS βάσει του service name (π.χ. `redpanda:9092`, `mongo:27017`). Ο Redpanda εκθέτει επιπλέον έναν εξωτερικό listener στο `localhost:19092` για host-side εργαλεία (`rpk`, Compass). Η MongoDB είναι mapped στο `27018:27017` στον host ώστε να μη συγκρούεται με τυχόν τοπική εγκατάσταση MongoDB.

Δικαιολόγηση επιλογών. Επιλέχθηκε ο Redpanda αντί για κλασική εγκατάσταση Kafka γιατί είναι μονό binary χωρίς JVM/ZooKeeper, μειώνοντας τους πόρους και τον χρόνο εκκίνησης ενώ διατηρεί πλήρη συμβατότητα στο wire protocol. Για το streaming layer προτιμήθηκε το Spark Structured Streaming αντί για το παλαιότερο Spark Streaming/DStreams, καθώς προσφέρει δηλωτικό DataFrame API και exactly-once σημασιολογία στους sinks. Η MongoDB διαλέχθηκε ως document store γιατί το payload του UXSIM είναι ήδη

Service	Ρόλος	Ports (host)
`redpanda`	Kafka-API broker (single node)	9092, 19092, 9644
`topic-init`	One-shot job: δημιουργεί topic `UXSIM`	—
`UXSIM-producer`	UXSIM simulation + Kafka publisher	—
`spark-master`	Spark standalone master	7077, 8080
`spark-worker`	Spark worker	8081
`spark-job`	Driver που τρέχει το consumer	4040
`mongo`	MongoDB Community 7.0	27018
`traffic-api`	FastAPI REST (`--profile api`)	8000
`dashboard`	Streamlit + Plotly (`--profile dashboard`)	8501

Πίνακας 1: Υπηρεσίες του `docker-compose.yml`.

ημι-δομημένο JSON και δεν απαιτεί ρητό σχήμα στη βάση.

3. Σχεδιαστικές Αποφάσεις

Functional core, OOP shell. Όλα τα aggregations είναι καθαρές συναρτήσεις `DataFrame` → `DataFrame` χωρίς πλευρενέργειες (`compute_link_stats`, `compute_windowed_stats`). Στιγμιότυπα κλάσεων χρησιμοποιούνται μόνο όπου υπάρχει πραγματικός κύκλος ζωής: `KafkaProducer`, `SparkSession`, `TrafficRepository`. Όλη η είσοδος/έξοδος (Kafka read, Mongo write, REST handlers) ζει στις άκρες (`scripts/`, `ingestion/`, `serving/`).

Output mode ανά sink. Η επιλογή `outputMode` έγινε χωριστά για κάθε query: το `raw_data` sink χρησιμοποιεί `append` (δεν υπάρχει ομαδοποίηση)· το `stats` sink χρησιμοποιεί `update` επειδή το `groupBy(t, link)` είναι αόριστο στο χρόνο και δε θέλουμε late-event drops· το `stats_windowed` sink χρησιμοποιεί `append` με `withWatermark("t", "60 seconds")` ώστε να εκπέμπει closed windows. Κάθε query έχει τον δικό του φάκελο checkpoint, όπως απαιτεί ο Spark.

Configuration layering. Οι παράμετροι ακολουθούν αυστηρά την προτεραιότητα YAML (`config/default.yaml`) < environment variables < CLI flags. Στο Spark container το YAML είναι μη διαθέσιμο, οπότε τα env vars είναι η authoritative πηγή εκεί. Τίποτα δεν είναι hard-coded μέσα στον κώδικα (broker addresses, topic name, collection names, simulation seed).

Πλήρης γραμμή στο schema. Ο UXSIM εκθέτει status rows με `link` ίσο με `waiting_at_origin_node` ή `trip_end`. Διατηρούνται ως έχουν στον parser και φιλτράρονται *μέσα* στη συνάρτηση aggregation με παράμετρο, ώστε τα tests να μπορούν να την καλέσουν με κενό σύνολο φίλτρων χωρίς duplication.

Σημείο προσοχής στη γλώσσα του πεδίου. Η εκφώνηση αναφέρεται στις στήλες ως `speed` και `time`. Ο UXSIM εκθέτει στην πραγματικότητα `v` (km/h) και `t` (seconds since simulation start). Η σύμβαση που ακολουθήθηκε είναι να αλιεύεται η αυθεντική στήλη του simulator και το rename σε `vspeed`/`vcount` να γίνεται μόνο στο επίπεδο του aggregation, ώστε να συμπίπτει με τους όρους της εκφώνησης στο τελικό sink.

4. Υλοποίηση ανά Ερώτημα

4.1 Ερώτημα 1: UXSIM → Redpanda

Το `ingestion/traffic_simulator.py` τρέχει έναν δομημένο 4×4 κόμβο road network (`ingestion/UXSIM_network.py`), προχωράει την προσομοίωση ανά step και καλεί την `World.analyzer.vehicles_to_pandas()` για να πάρει το τρέχον στιγμιότυπο οχημάτων. Το `ingestion/kafka_publisher.py` σειριοποιεί κάθε γραμμή ως JSON και δημοσιεύει στο topic `UXSIM`, με exponential backoff σε αποτυχίες σύνδεσης. Η παράμετρος `PRODUCE_INTERVAL_SEC` καθορίζει το wall-clock βήμα μεταξύ δύο simulation steps. Το topic δημιουργείται αυτόματα από το one-shot service `topic-init`, ο producer εξαρτάται μέσω `depends_on: service_completed_successfully` ώστε να εξαλείφεται το race condition του πρώτου boot.

4.2 Ερώτημα 2: Spark Structured Streaming consumer

Το `preprocessing/schemas.py` ορίζει το `UXSIM_EVENT_SCHEMA` (9 πεδία: `name`, `dn`, `orig`, `dest`, `t`, `link`, `x`, `s`, `v`). Η pure συνάρτηση `parse_kafka_stream` κάνει `from_json` πάνω στο binary `value` του Kafka source και επιστρέφει τυποποιημένο DataFrame. Το aggregation στο `models/link_stats.py` είναι:

```
parsed.groupBy("t", "link").agg(count("*").alias("vcount"), avg("v").alias("vspeed"))
```

Με `outputMode("update")` κάθε batch εκπέμπει μόνο τα κλειδιά `(t, link)` των οποίων η συγκεντρωτική μετρική άλλαξε. Για debugging είναι δυνατό να ενεργοποιηθεί παράλληλο console sink θέτοντας `SPARK_DEBUG_CONSOLE=1`.

4.3 Ερώτημα 3: MongoDB sinks

Δύο writers χρησιμοποιούν `foreachBatch` με `mode("append")`: ο πρώτος γράφει την επεξεργασμένη ροή στη συλλογή `traffic.raw_data` (ένα έγγραφο ανά event), ο δεύτερος γράφει τα aggregates στη `traffic.stats` (ένα έγγραφο ανά `(t, link)`). Η σύνδεση γίνεται μέσω του επίσημου MongoDB Spark Connector v10.3.0 για Scala 2.12 (συμβατός με Spark 3.5.2). Όλη η μη streaming πρόσβαση (REST handlers, tests) γίνεται μέσω της κλάσης `TrafficRepository` που τυλίγει έναν shared `MongoClient`.

4.4 Bonus που υλοποιήθηκαν

Windowed aggregations (30s). Η `compute_windowed_stats` εφαρμόζει `withWatermark("t", "60 seconds")` και `window("t", "30 seconds")`, υπολογίζοντας `vcount`, `vspeed_avg`, `vspeed_min`, `vspeed_max`. Το νεστάρισμα του struct ξεδιπλώνεται πριν το sink, ώστε τα έγγραφα της `stats_windowed` να έχουν επίπεδα πεδία `window_start` και `window_end` (ευκολότερο indexing και query από REST).

REST API. Το `serving/api.py` σε FastAPI εκθέτει `GET /health`, `GET /links/avg-speed`, `GET /links/top-congested?n=K` και `GET /links/{link_id}`. Ένα singleton `MongoClient` αρχικοποιείται μέσω lifespan event handler και γίνεται inject στους handlers με `Depends`. Με ενεργό το `--profile api`, η αυτο-παραγόμενη διεπαφή Swagger UI είναι διαθέσιμη στο http://localhost:8000/docs, επιτρέποντας διαδραστική εκτέλεση κάθε endpoint από τον browser. Το `link_id` είναι το όνομα ενός edge στο 4×4 grid με μορφή `<origin><destination>`, π.χ. `I1W1`, `I3S3`, `N1I1`, `E1I4`, `I2I1` (όπου `I1..I4` οι εσωτερικές διασταυρώσεις και `N*/S*/E*/W*` οι συνοριακοί κόμβοι).

Streamlit dashboard. Τρία tabs (Overview, Network map, Link detail) που διαβάζουν απευθείας από MongoDB μέσω της ίδιας `TrafficRepository`. Ο χάρτης είναι μία ζωντανή απεικόνιση Plotly του 4×4 δικτύου με χρώμα ίσο με `vspeed` και πάχος γραμμής ίσο με `vcount`, με slider για historical playback.

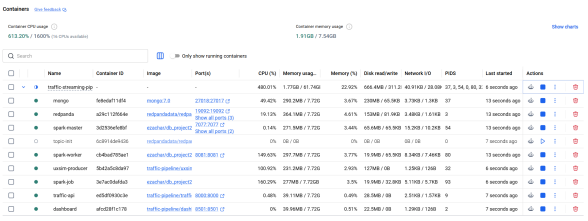
5. Επαλήθευση και Ενδεικτικά Αποτελέσματα

Η σουίτα unit tests (`tests/unit/`, pytest) καλύπτει τον parser, τα δύο aggregations, τον Kafka publisher με mocked broker και το repository με `MagicMock` (25 tests, δεν απαιτεί running stack). Η ολοκληρωτική επαλήθευση γίνεται από το `scripts/verify_pipeline.py`, που εκτελεί τέσσερις ελέγχους: (1) όλα τα containers είναι Up, (2) το topic `UXSIM` υπάρχει, (3) η `raw_data` γεμίζει, (4) η `stats` γεμίζει. Σε ενδεικτικό clean run διάρκειας λίγων λεπτών συγκεντρώθηκαν περίπου 590 000 έγγραφα στη `raw_data` και 7 000 στη `stats`. Η ζητούμενη από την εκφώνηση συγκεντρωτική ερώτηση (μέση ταχύτητα ανά link) εκτελείται απευθείας στο `root terminal`:

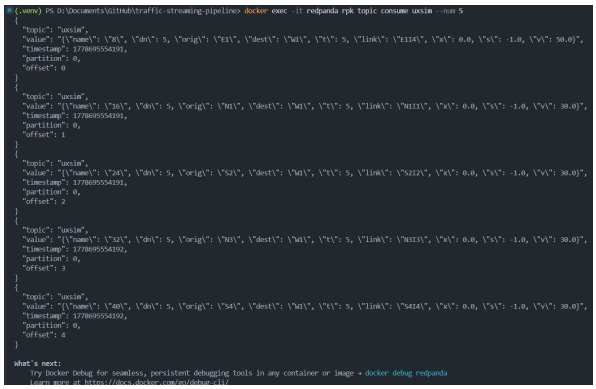
```
docker exec mongo mongosh traffic --quiet --eval 'db.getCollection(\"stats\").aggregate([{
$group: { _id: \"$link\", avg_speed: { $avg: \"$vspeed\" } } }, { $sort: { avg_speed: -1 }
                                    }]).toArray()'
```

5.1 Screenshots από την τοπική εκτέλεση

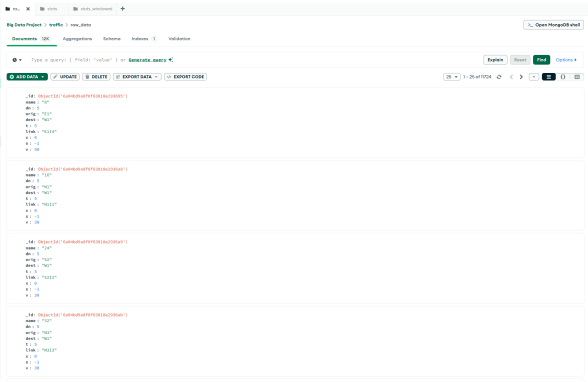
Η εκφώνηση ζητάει screenshots από Upstash, MongoDB, Spark console και ένα aggregation query. Επειδή η υλοποίηση είναι 100% τοπική σε Docker, τα αντίστοιχα στιγμιότυπα προέρχονται από εργαλεία host-side (Docker Desktop, MongoDB Compass, Streamlit) αντί για cloud UI. Τα αρχεία βρίσκονται στο `docs/img/`.



Σχήμα 2: Docker Desktop, καρτέλα Containers με όλα τα services της στοίβας σε κατάσταση *Running* (ισοδύναμο του Upstash dashboard).



Σχήμα 3: Έξοδος του `rpk topic consume` μέσα στο container Redpanda, με δείγμα μηνυμάτων JSON (ισοδύναμο του Upstash "Messages" tab).



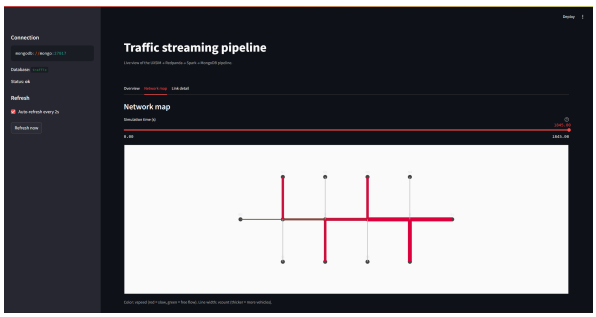
Σχήμα 4: MongoDB Compass συνδεδεμένο στο `mongodb://localhost:27018`, με τις συλλογές `raw_data`, `stats` και `stats_windowed` της βάσης `traffic`.



Σχήμα 5: Η αναζητούμενη μέση ταχύτητα ανά link, εκτελεσμένη στο Aggregation tab του Compass (ισοδύναμα στο `mongosh`).



Σχήμα 6: `docker logs -f spark-job` με τα μηνύματα "Batch processed" του Structured Streaming (ενδεικτικά output του Spark console sink).



Σχήμα 7: Streamlit dashboard στο `http://localhost:8501`, καρτέλα *Network map*, με χρωματισμό ακμών κατά `vspeed` (bonus παρουσίαση των δεδομένων).

6. Προβλήματα που Αντιμετωπίστηκαν

- **`kafka-python` 2.0.2 incompatibility με Python 3.12** (`ModuleNotFoundError: kafka.vendor.six.moves`). Λύση: pin σε `kafka-python-ng 2.2.3`, που είναι API-συμβατό fork.
- **Race condition στο πρώτο boot**, όπου ο producer ξεκινούσε πριν δημιουργηθεί το topic. Λύση: ξεχωριστό one-shot service `topic-init` και εξάρτηση μέσω `depends_on: service_completed_successfully`.
- **Σύγκρουση πορτών MongoDB** στο host (τυπική τοπική εγκατάσταση στο 27017). Λύση: mapping σε `27018:27017` και ενημέρωση του `MONGO_URI_HOST`.

- **Windows + `spark.createDataFrame` sockets**, που πάγωναν σε integration tests. Λύση: αντικατάσταση των in-process Python rows με `spark.sql("VALUES ...")`, ώστε όλη η δουλειά να μένει στη JVM.

7. Συμπεράσματα και Μελλοντικές Επεκτάσεις

Παραδίδεται ένα πλήρες, αναπαραγώγιμο pipeline που εκκινείται με μία εντολή Docker Compose, καλύπτει και τα τρία ερωτήματα της εκφώνησης και επεκτείνει το ζητούμενο με windowed aggregations, REST API και dashboard. Πιθανές μελλοντικές βελτιώσεις: idempotent upserts στη `stats` με σύνθετο `_id = (t, link)` για ασφαλή επανεκτέλεση, alerting όταν η `vspeed` ενός link πέφτει κάτω από κατώφλι, μετακίνηση των checkpoints από `/tmp/` σε named volume για επιβίωση μετά από restart του cluster, και παραμετροποίηση της τοπολογίας του δικτύου από YAML αντί από Python literal.